UNIVERSITÉ DE CAEN NORMANDIE

PROJET DE PROGRAMMATION AVANCÉE

IMPLÉMENTATION D'UN JEU DE BLACKJACK

Auteurs :

Aissani Ciril

Mezine Lounes

Gustavo Vieira Dos Santos

Ahmed Bezeid Mohamed El Mamy

Encadrant :

Yann Mathet

5 décembre 2025

Année Académique : 2025 - 2026

Table des matières

1 Introduction et Architecture du Projet **2**
1.1 Principes Architecturaux 2
1.1.1 Modèle-Vue-Contrôleur (MVC) 2
1.1.2 Motif Observateur/Observable 2

2 Diagramme de Classes UML **3**
2.1 Relations Clés 3

3 Analyse Détaillée des Composants **4**
3.1 Le Modèle : Logique du Jeu et Gestion des Cartes 4
3.1.1 Logique du Blackjack (`blackjack.modele`) 4
3.1.2 Fondation de Cartes (`cartes.modele` et `cartes.util`) 4
3.2 Les Vues et Contrôleurs : Interfaces Utilisateur 4
3.2.1 Vues de Jeu Spécifiques (`blackjack.vue`) 4
3.2.2 Contrôleurs de Jeu (`blackjack.controleur`) 4

4 Problèmes Rencontrés et Solutions **5**
4.1 Gestion des Entrées-Sorties en Console 5
4.1.1 Problème avec `scanner.close()` 5
4.2 Intégration du Modèle de Cartes 5
4.2.1 Importation des Classes Modèle en Temps Réel 5
4.3 Implémentation de l'Interface Graphique 6
4.3.1 Difficultés de Style et de Rendu des Cartes 6

5 Conclusion Finale **7**
5.1 Validation Architecturale et Synthèse 7
5.2 Perspectives d'Évolution 7

Chapitre 1

Introduction et Architecture du Projet

Ce rapport documente l'architecture et l'implémentation d'un jeu de Blackjack en Java, basé sur le motif **Modèle-Vue-Contrôleur (MVC)**, complété par le motif **Observateur/Observable**.

1.1 Principes Architecturaux

1.1.1 Modèle-Vue-Contrôleur (MVC)

Le projet sépare strictement la Logique Métier (Modèle), l'Affichage (Vue) et les Interactions (Contrôleur).

1.1.2 Motif Observateur/Observable

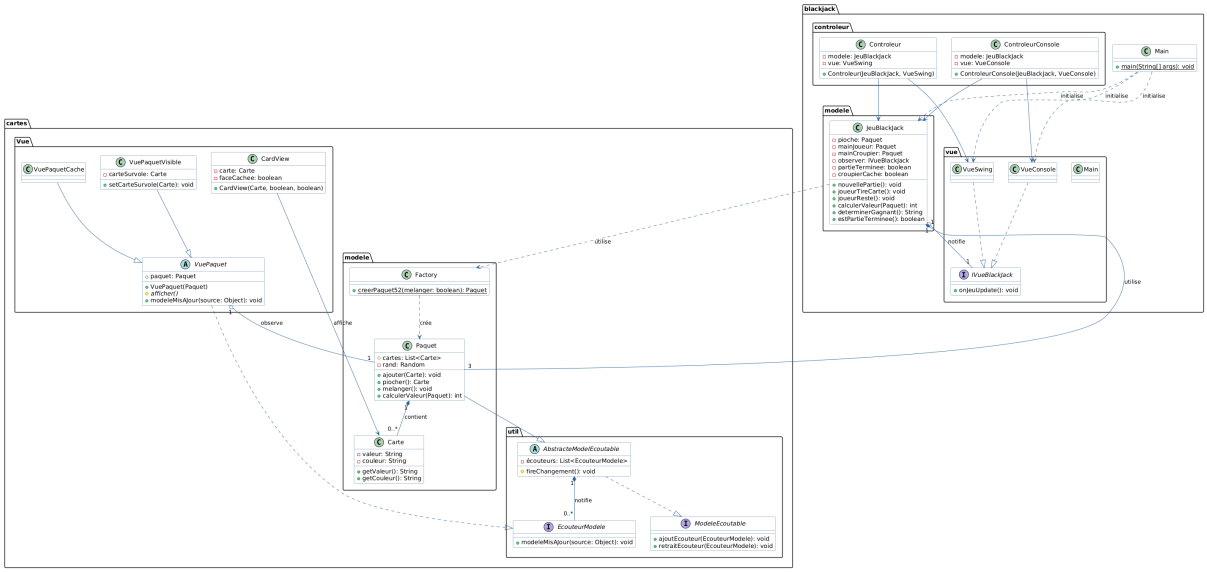
Le Modèle (`Paquet`, `JeuBlackJack`) est un Observable qui notifie les Vues de tout changement pour garantir une réactivité automatique de l'interface.

Chapitre 2

Diagramme de Classes UML

La figure 2.1 illustre l'intégralité de la structure logicielle, couvrant la fondation générique et la logique spécifique au Blackjack.

FIGURE 2.1 – Diagramme de Classes UML Complet du Projet Blackjack



2.1 RELATIONS CLÉS

— **Héritage d'Observable Général :** `Paquet` ↑ `AbstracteModelEcoutable`.

— **Implémentation d'Observateur Spécifique :** `VueSwing` / `VueConsole` ↑ `IVueBlackJack`.

Chapitre 3

Analyse Détaillée des Composants

3.1 Le Modèle : Logique du Jeu et Gestion des Cartes

3.1.1 Logique du Blackjack (`blackjack.modele`)

La classe `JeuBlackJack` est l'épine dorsale du jeu. Elle implémente correctement la règle de l'As et gère le déroulement de la partie (distribution, tour du Croupier, détermination du vainqueur).

3.1.2 Fondation de Cartes (`cartes.modele` et `cartes.util`)

`Paquet` est le conteneur de cartes observable. La `Factory` assure l'instanciation normalisée du jeu de 52 cartes.

3.2 Les Vues et Contrôleurs : Interfaces Utilisateur

3.2.1 Vues de Jeu Spécifiques (`blackjack.vue`)

— `VueConsole` : Implémentation en mode texte.
— `VueSwing` : Implémentation graphique (Java Swing), gérant l'affichage dynamique des cartes (`CardView`).

3.2.2 Contrôleurs de Jeu (`blackjack.controleur`)

— `Controleur` **(Swing)** : Gère les événements des boutons Swing et le clic sur la pioche.
— `ControleurConsole` : Gère la boucle de jeu et les entrées clavier.

Chapitre 4

Problèmes Rencontrés et Solutions

L'implémentation du projet, bien que réussie, a nécessité la résolution de plusieurs défis techniques, particulièrement liés à la gestion des entrées-sorties et à la complexité de l'interface graphique.

4.1 Gestion des Entrées-Sorties en Console

4.1.1 Problème avec `scanner.close()`

Lors du développement du `ControleurConsole`, une difficulté majeure a été rencontrée lors de la gestion des ressources d'entrée standard (`System.in`). Tenter d'appeler `scanner.close()` après utilisation provoquait une fermeture du flux `System.in` sous-jacent.

- **Conséquence :** Si une autre partie du programme tentait d'accéder à `System.in` après sa fermeture, une erreur critique (`java.util.NoSuchElementException`) se produisait.
- **Solution :** Il a été décidé de laisser le flux ouvert jusqu'à l'arrêt complet du programme (`System.exit(0)`), ce qui est une pratique courante pour la gestion de l'entrée standard.

4.2 Intégration du Modèle de Cartes

4.2.1 Importation des Classes Modèle en Temps Réel

La séparation des packages (`cartes` et `blackjack`) a nécessité une gestion rigoureuse des dépendances et des imports pour assurer la cohérence entre le Modèle générique et la logique spécifique du Blackjack.

4.3 IMPLÉMENTATION DE L'INTERFACE GRAPHIQUE

4.3.1 Difficultés de Style et de Rendu des Cartes

L'implémentation de la `CardView` dans l'environnement Swing a posé des défis en matière de rendu (`paintComponent`) et de positionnement des composants (layout `null` et calcul manuel des coordonnées pour l'effet d'éventail).

Chapitre 5

Conclusion Finale

L'objectif de créer un jeu de Blackjack basé sur une architecture solide est pleinement atteint. Le projet démontre une maîtrise de l'architecture **MVC** et des motifs de conception pour créer un système **découplé** et **réactif**. Le Modèle `JeuBlackJack` est réutilisable, permettant aux deux Vues (`Swing` et `Console`) de fonctionner avec la même logique de jeu.

5.1 Validation Architecturale et Synthèse

L'architecture est validée par la création de deux interfaces utilisateur distinctes partageant le même Modèle. Le succès de l'intégration, malgré les difficultés rencontrées en I/O et en IHM, confirme la robustesse du design MVC.

5.2 Perspectives d'Évolution

— **Gestion des Mises :** Intégrer la gestion des jetons et des mises, et la fonctionnalité *Double Down*.
— **Amélioration Graphique :** Utiliser des images de cartes réelles pour la `CardView`.

Fin du Rapport